Bill Reminder System — PRD

Oct 29, 2026 · @ps

Overview

Version 2: multi-user. A web app and Telegram bot where anyone can sign up, add their recurring bills (electricity, gas, maintenance, mobile recharge, subscriptions), get reminders before each due date, and log payments from the website or Telegram using text, a pasted SMS or a screenshot.

Problem: bills run on different cycles (28-day recharges, monthly maintenance, yearly subscriptions) and live in people's heads. A missed date means a lapsed recharge or a late fee.

Users: anyone with an email address. Each user logs in to a private dashboard and links their own Telegram chat. Nobody can see another user's data. First users: Prisha and Vritti.

Goals and success criteria

Success means no user misses a bill and logging a payment takes under 20 seconds.

Goal	How we know it works
Never miss a due date	Alerts arrive at each user's chosen lead times, every time
Fast logging	Paid is one tap on web or Telegram; details by text, SMS paste or screenshot
Private by default	Every user sees only their own bills (row-level security)
Smart recurrence	A new bill asks how often it repeats, then schedules itself
Always on, free	Runs on Vercel Hobby, Supabase free, Telegram and Gemini free tier
Attractive and fast	Bento-style, Gen Z-friendly UI with micro-interactions that never slow the page
Portfolio-ready	Live Vercel URL, GitHub repo, README, short demo video

Scope

Version	Included
MVP (3 days)	Sign-up and login; Telegram linking with a one-time code; add a bill by form, text, SMS paste or screenshot; recurrence and reminder settings; daily reminders; Paid / Snooze / Ignore on web and Telegram; glass-style bento dashboard with charts; monthly expense tracker (built from payments); Excel/CSV export
Stretch	Optional monthly budget with a progress ring and warning colours
Later	Yearly view, shared household bills, per-user reminder time, installable PWA, email alerts
Out of scope	Reading bank SMS automatically, making payments, comparing plans or prices, paid plans, native mobile app

Logging stays semi-manual on purpose: automation covers parsing, recurrence, reminders and tracking.

User flows

Sign up and link Telegram

1. Sign up on the website with email and password.
2. The dashboard shows a one-time code.
3. Send `/start CODE` to the bot.
4. The bot confirms. From now on alerts, Paid buttons and uploads for this account go to that chat.

Add a bill (three ways, one shared list)

- Website form.
- Telegram text, such as "Vodafone recharge done, 28 days validity".
- A pasted SMS or email, or a screenshot, sent to the bot or uploaded on the website.

Gemini reads the input, the app shows what it understood, and then asks the recurrence and reminder questions in the next section.

Daily reminder: every morning the system checks all users' bills and alerts each user at the lead times they chose. Each alert has Paid, Snooze and Ignore buttons.

Mark as paid, on the website

1. Tap Paid on a bill card.
2. A small panel opens with the amount and date prefilled.
3. Confirm, or edit, or attach an SMS or screenshot.
4. The card animates to Paid and the next due date appears.

Mark as paid, on Telegram

1. Tap Paid on the alert. The bot asks for details.
2. Send text, a pasted SMS or email, or a screenshot.
3. The bot confirms the amount, date and next due date.

Both routes update the same record, so web and Telegram always agree.

Snooze re-alerts the next day. **Ignore** stops alerts for this cycle, but the bill stays Overdue on the dashboard.

Export: one button downloads the user's bills and payments as an Excel/CSV file.

Recurring bills and reminder settings

Every bill gets two settings, asked when it is added and editable later: how often it repeats, and when to remind. Account-level defaults apply unless a bill overrides them.

Two ways to start

Path	How it works
Set up first	A Settings wizard lists common bills (electricity, gas, maintenance, recharge, subscriptions); the user ticks the recurring ones and sets cycle and amount
Add as you pay	The user logs this month's payment; the app asks "Repeat this?" and schedules next month's bill automatically

How often does it repeat?

Option	Behavior
Use what I sent	Uses the cycle found in the message, e.g. a recharge with 28-day validity
Every month	Same day each month
Every N days	Custom number of days
Every N months	Custom number of months
Quarterly or yearly	Fixed 3 or 12 months
One-time	No next due date

When should I remind you?

Option	Example for a 28-day recharge
1 day before	Alert on day 27
2 days before	Alert on day 26
3 days before	Alert on day 25
On the due day	Alert on day 28
Custom	User picks any days before; several allowed

On Telegram these questions are asked as buttons right after a bill is saved. On the website they appear in the add-bill panel and in Settings. The default is 3 days, 1 day and the due day.

Functional requirements

ID	Requirement
F1	Email and password sign-up and login (Supabase Auth)
F2	Telegram linking with a single-use, expiring code
F3	Add a bill by form, text, SMS paste or screenshot
F4	Gemini parses input into a fixed schema: biller, amount, date, cycle or validity, reference, confidence
F5	Recurrence and reminder settings asked for each new bill, editable later, with account defaults
F6	Next due date = paid date + cycle; a validity found in the message overrides the default
F7	Daily scheduled job sends reminders for every user at their lead times
F8	Paid, Snooze and Ignore work on web and Telegram and stay in sync
F9	Duplicate payments rejected by transaction reference
F10	Export own data to Excel/CSV

ID	Requirement
F11	Per-user daily limit on AI parsing
F12	Delete account and all data

Edge cases

Case	Behavior
Unlinked person messages the bot	Bot replies with how to link an account
Screenshot sent without tapping Paid	Match biller name to one of the user's bills
Low confidence or no match	Ask "Is this for Vi Recharge? Yes / No"
Amount differs from usual	Log it and update the bill's amount
Gemini fails or times out	Ask for manual text; never lose the message
Linking code expired or reused	Reject and show a fresh code

Data model

Supabase Postgres. Every table except link_codes carries user_id, and row-level security lets a user read and write only rows where user_id is theirs.

profiles

Column	Example
user_id	(from Supabase Auth)
display_name	Prisha
telegram_chat_id	123456789
default_remind_days	3,1,0
created_at	2026-10-31

link_codes

Column	Example
code	K7P2QX

Column	Example
user_id	(owner of the code)
expires_at	2026-10-31 10:15
used	false

bills

Column	Example
bill_id	B001
user_id	(owner)
name	Vi Recharge
category	recharge
icon	phone
amount	299
cycle_type	days / months / one-time
cycle_value	28
last_paid_date	2026-10-31
next_due_date	2026-11-28
remind_days	1
snoozed_until	2026-11-27
status	active / ignored / paused

payments

Column	Example
payment_id	P001
user_id	(owner)
bill_id	B001
paid_date	2026-10-31
amount_paid	299

Column	Example
txn_ref	UPI123456
source	web / telegram
input_type	form / text / sms / screenshot

bot_state

Column	Example
chat_id	123456789
user_id	(owner)
waiting_for	paid_details
bill_id	B001
updated_at	2026-10-31 17:45

ai_usage

Column	Example
user_id	(owner)
day	2026-10-31
count	4

Dashboard requirements

A Next.js site on Vercel, designed in Antigravity. After login each user sees only their own data.

Page	Shows
Welcome	One bold line (for example "Never miss a bill again") over a glass card with two buttons: Create account and Log in
Home	Bento grid: next due, overdue, paid this month, monthly spend chart, upcoming list, recent payments, quick add

Page	Shows
Bills	All bills with cycle, last paid, next due, reminder settings; edit and pause
Expenses	Monthly expense tracker: total spent this month, split by category (donut), month-by-month bars, and a change versus last month. Built from the payments table, so it needs no extra input. Stretch: an optional monthly budget with a progress ring that turns amber near the limit and red over it
History	Searchable, filterable list of payments with a month picker
Settings	Telegram linking, default reminders, recurring-bill wizard, export, delete account

New users land on a short onboarding: link Telegram, then choose "Set up recurring bills" or "Add as I pay". A bottom action bar on mobile gives quick access to Add Bill, Expenses and Set Reminder.

Design direction

The site should feel modern, playful and Gen Z friendly, not like a dull finance dashboard, and it must stay fast. Antigravity should treat every row below as a requirement and explain its choices in code comments.

What to take from the references. Use them for layout and feel only. Do not copy artwork, logos, icons or wording.

Reference	Take this
Dark finance app with frosted cards	Translucent glass cards over a soft gradient background; large bold numbers; a bar chart as the hero; rounded colour-coded icon tiles on each list row; a bottom action bar; a welcome screen with one statement and two buttons
Vi website "explore more services" row	Rounded square icon tiles, each with a simple line icon and a short label underneath, so people recognise an action by its icon first

Requirements

Area	Requirement
Style	"Aurora glass": deep navy base with soft blurred gradient glows (violet, blue, teal) behind frosted glass cards. Cards use a translucent fill, a light blur and a thin light border. Keep blur small and layers few for speed; fall back to a solid translucent colour where blur is unsupported
Data over text	Every card leads with a number or a chart and carries at most one short line of text. Bars for monthly spend, donut for categories, progress ring for days until due, small sparklines on bill cards. Details appear on tap or hover
Layout	Bento grid of rounded cards in mixed sizes. "Due next" is the large hero card with a violet-to-blue gradient. Mobile first: cards stack into one column on a phone
Icons	One distinct line icon per bill type in a rounded tile with a soft tint and a label beneath, as in the Vi row. Use inline SVG, one consistent stroke width. The same types map to emoji in Telegram alerts
Micro-interactions	Paid button morphs into a checkmark; totals count up; charts draw in once; cards lift slightly on hover or press; swipe on mobile to mark Paid or Snooze; toasts include Undo; skeleton shimmer while loading; slow drift on the background glows
Motion rules	Animate only transform and opacity; keep each animation short; respect the reduced-motion setting; no heavy animation libraries
Speed	Server-rendered pages, inline SVG icons, at most one subsetted font, lazy-load below the fold, no large images (glows are CSS gradients, not image files). Target a mobile Lighthouse performance score of 90 or higher
Readability	Text on glass must keep strong contrast; test every card in light and dark
Tone	Friendly microcopy and playful empty states

Area	Requirement
Themes	Dark by default (glass looks best on it), with a light option following the device setting

Bill type icons

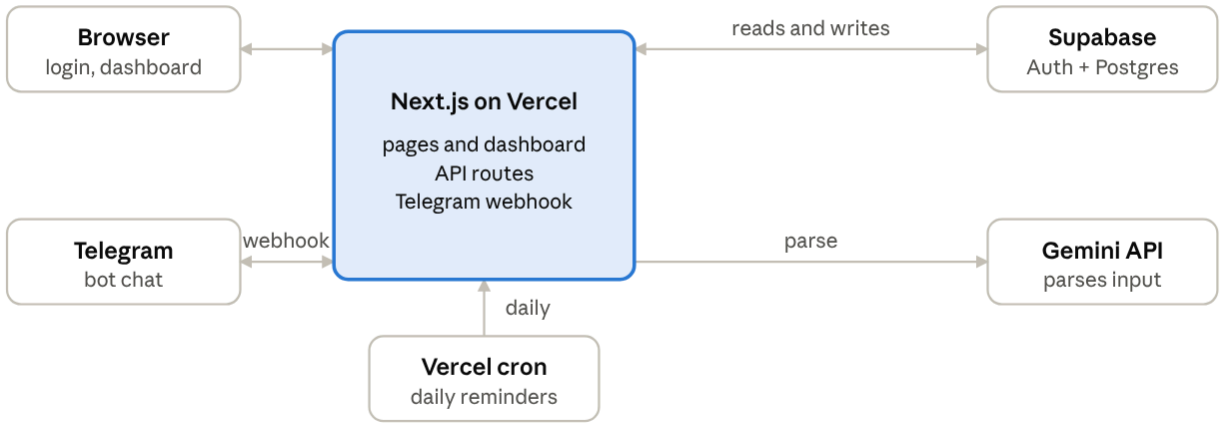
Type	Icon idea	Tile tint
Electricity	Lightning bolt	Amber
Gas	Flame	Coral
Water	Droplet	Blue
Maintenance	Building	Violet
Mobile recharge	Phone with signal	Teal
Internet	Wi-Fi	Sky
Subscription	Play button or clapper	Pink
Other	Receipt	Grey

Palette (proposed; replace with Prisha's final choices)

Role	Colour
Background	Midnight navy #0B1026
Glass surface	White at 8% opacity, border white at 14%
Primary	Violet #7C5CFF
Accent	Electric blue #4C8DFF
Glow	Aurora teal #2DE2C7 (background blobs and highlights)
Paid	Mint #3DDC97
Due soon	Amber #FFB84D
Overdue	Coral #FF5D73
Text	#F4F6FF, secondary #A9B0D0

Architecture and tech stack

One Next.js app serves the website, the bot and the daily reminders



system architecture · 6 parts

Every request goes through one Next.js app on Vercel: the website, the Telegram webhook and the daily reminder job. Supabase handles login and stores each user's data; Gemini only parses input.

Layer	Tool	Cost
Website	Next.js, designed in Antigravity	Free
Hosting	Vercel Hobby	Free (non-commercial use)
Login and database	Supabase Auth and Postgres	Free tier; projects pause after a week idle, and the daily reminder job should count as activity
Bot	Telegram Bot API	Free
Parsing	Gemini API	Free tier, rate-limited
Scheduler	Vercel cron	Free; once a day on Hobby
Code	GitHub	Free

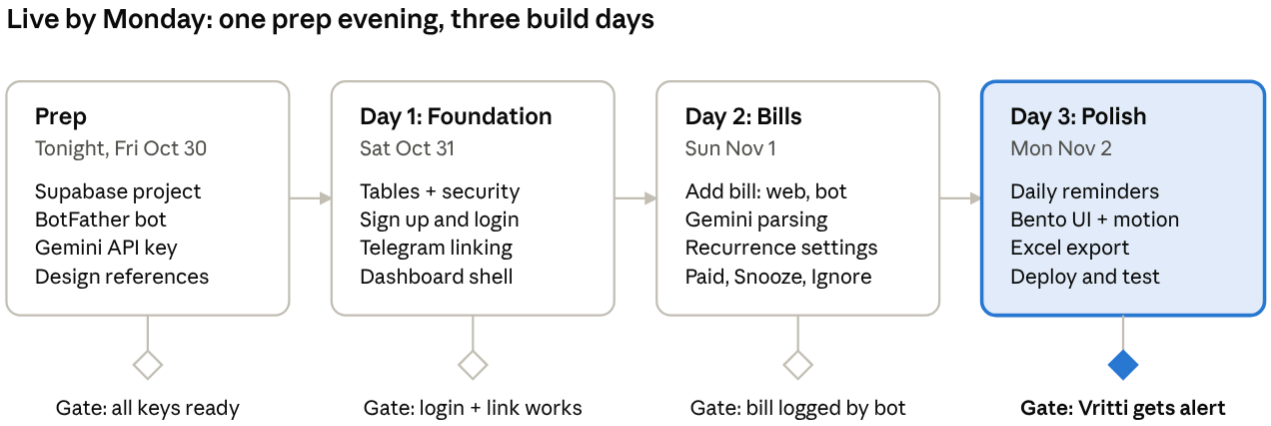
Security and privacy

Other people's bills are personal data, so these are requirements, not extras.

- Supabase Auth handles passwords; the app never stores or logs them.

- Row-level security on every table: a user can only read and write their own rows.
- Secrets (Telegram token, Gemini key, Supabase service key) live in Vercel environment variables, never in browser code or the GitHub repo.
- The Telegram webhook checks Telegram's secret header, and only linked chat IDs can act on an account.
- Linking codes are single-use and expire.
- The cron endpoint requires a secret.
- Screenshots are parsed and discarded; only extracted fields are saved. Card and bank account numbers are never stored.
- A privacy note on the sign-up page says what is stored. Before inviting others, read Google's terms on how free-tier Gemini data is used.
- Per-user daily cap on AI parsing; sign-up is rate-limited.
- Deleting an account removes all its data.

Timeline



build plan · 3 phases, 3 gates

Do the prep tonight, build over Saturday to Monday, and go live Monday. Each gate must pass before the next phase starts. Vritti is the second test user on Day 3. Start the scam-call project after launch so the two builds don't overlap.

Risks and open questions

Risk	Fallback
Supabase free project pauses after a week idle	Daily reminder job as activity; check after week one; keep the export button as a backup
Vercel Hobby cron runs once a day, anywhere in the hour	Fine for bills; per-user reminder times later need a paid plan or an external scheduler
Free Gemini limits are shared by all users	Per-user daily cap; manual form as fallback
Gemini misreads a screenshot	Confirm step and manual edit
Storing other people's data	Minimal storage, privacy note, delete account
Vercel Hobby is for non-commercial use	Keep the app free; revisit if it is ever monetised

Open questions

- [ ] Final project name
- [ ] Colour palette and reference designs from Prisha
- [ ] Default reminder days (suggested: 3, 1 and the due day)
- [ ] Bill categories and icons to ship first
- [ ] Privacy note wording